

# INTEGRATED PLATFORM FOR CROWDSOURCED OCEAN HAZARD REPORTING AND SOCIAL MEDIA ANALYTICS

DONE BY:

Afeia Justin Jose Little Bright (192524494)

Roshnee Devi Kannan (192511002)

Anisha Jenifer J (192511004)

## 1. INTRODUCTION

India has an extensive coastline stretching over thousands of kilometers, making it highly vulnerable to a wide range of ocean-related hazards such as tsunamis, storm surges, high waves, coastal flooding, and strong currents. These hazards can cause significant damage to coastal infrastructure, disrupt livelihoods, and pose serious threats to human life.

Organizations such as the Indian National Centre for Ocean Information Services (INCOIS) play a vital role in monitoring ocean conditions and issuing early warnings using advanced technologies like satellite imaging, ocean buoys, and numerical models. These systems are highly reliable and scientific in nature; however, they primarily depend on instrument-based data collection.

Despite these advancements, there remains a gap in capturing real-time, ground-level observations from people who are directly affected. Coastal residents, fishermen, and local communities often notice unusual sea behavior—such as sudden rise in water levels, abnormal tides, or strong currents—before such data is formally recorded.

In addition to this, social media platforms have emerged as powerful tools for real-time information sharing. During disaster situations, people actively post updates, warnings, and experiences. However, this information remains largely unstructured and is not effectively utilized for disaster management.

This project proposes a C++-based system that integrates crowdsourced reporting and text-based analysis to simulate how such real-time information can be collected, processed, and used to improve disaster awareness and response.

## 2. OBJECTIVES

The primary objective of this project is to design and implement a simplified yet effective disaster reporting and analysis system using C++. The system aims to demonstrate how real-time data from multiple sources can be processed to identify potential hazard situations.

The specific objectives include:

- To create a system that allows users to input and store information about ocean-related hazards such as location, type of hazard, and description.

- To process textual data and identify important keywords related to disaster events using string manipulation techniques in C++.
- To analyze multiple reports and detect patterns, such as repeated occurrences of similar hazards in a specific location.
- To simulate the concept of social media analytics by processing sample text inputs representing public posts and discussions.
- To identify high-risk or hotspot areas based on the frequency and type of reported hazards.
- To demonstrate the use of core C++ concepts such as structures, arrays, file handling, and algorithms in solving real-world problems.
- To build a foundation that can be extended into a more advanced system with graphical interfaces and real-time data integration.

### **3. PROBLEM STATEMENT**

Disaster management systems today rely heavily on data collected from sensors, satellites, and predictive models. While these technologies are accurate and essential, they often lack immediate feedback from ground-level conditions. This delay in capturing real-time human observations can lead to slower response times during critical situations.

Furthermore, during disaster events, social media platforms become highly active with users sharing information, warnings, and personal experiences. This data contains valuable insights into the severity of the situation, affected areas, and public sentiment. However, due to the unstructured nature of this data and the absence of proper analytical tools, much of this information remains unused.

Another major challenge is the absence of a unified system that combines official data sources with citizen-generated information. Without such integration, disaster response teams may not have a complete understanding of the situation on the ground.

Therefore, the problem addressed in this project is the lack of an efficient system that:

- Collects real-time hazard information from the public
- Processes and analyzes textual data effectively
- Identifies patterns and critical zones quickly
- Provides meaningful insights for decision-making

This project aims to address these challenges by developing a C++-based system that simulates crowdsourced reporting and basic data analysis to identify potential hazard zones.

### **4. PROPOSED SYSTEM**

The proposed system is a console-based application developed using C++ that allows users to input hazard-related data and processes this information to generate useful insights. The system acts as a simplified model of a real-world disaster reporting platform.

Users can enter details such as location, type of hazard (e.g., flood, high waves, tsunami), and a brief description. The system stores this data and performs analysis using basic algorithms and string processing techniques.

In addition, the system simulates social media analysis by accepting text inputs that represent public posts. These inputs are analyzed to detect relevant keywords and determine whether they indicate a potential hazard situation.

Based on the collected data, the system identifies areas with a high number of reports and classifies them as potential risk zones or hotspots. The results are then displayed to the user in a structured format.

This system demonstrates how simple programming concepts in C++ can be applied to solve real-world problems related to disaster management.

## **5. SYSTEM ARCHITECTURE**

The system is designed in a layered structure to ensure clarity and modularity.

### **5.1 Input Layer**

This layer is responsible for collecting data from users. It includes inputs such as location, type of hazard, and descriptive text. It also accepts simulated social media posts for analysis.

### **5.2 Processing Layer (C++ Core)**

This is the main component of the system where all computations take place. It includes:

- Data validation
- Keyword detection using string operations
- Classification of hazards
- Frequency analysis for identifying hotspots

### **5.3 Data Storage Layer**

The system stores data using C++ structures, arrays, or file handling techniques. This ensures that multiple reports can be recorded and analyzed collectively.

### **5.4 Output Layer**

The output layer presents the processed information to the user. It displays hazard details, detected keywords, and identified high-risk zones in a clear and readable format.

## **6. TECHNOLOGY STACK**

The project is implemented using the C++ programming language, focusing on fundamental programming concepts to simulate a real-world system.

Key technologies and concepts used include:

- C++ programming language
- Structures and classes for data organization
- Arrays and vectors for storing multiple records
- File handling for persistent data storage
- String processing for keyword detection
- Basic algorithms for pattern recognition and analysis

## **7. WORKING OF THE SYSTEM**

The system operates in a step-by-step manner:

Initially, the user provides input regarding a hazard event, including location, type, and description. This data is stored in the system using appropriate data structures.

Next, the system processes the textual description to identify important keywords related to disaster events. This is done using string comparison and searching techniques.

The system then analyzes all collected reports to determine if multiple entries correspond to the same location or hazard type. If a location has a high number of similar reports, it is flagged as a potential risk zone.

Additionally, simulated social media text is analyzed to detect similar keywords and trends. This helps in understanding how public discussions can indicate emerging hazards.

Finally, the system displays the analyzed results, including identified hotspots and relevant hazard information.

## **8. PROGRAM**

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
struct Report {
```

```
    string location;
```

```
    string hazardType;
```

```

    string description;
};

int main() {
    int n;

    cout << "Enter number of reports: ";
    cin >> n;
    cin.ignore(); // clear buffer

    Report reports[100];

    // Input reports
    for (int i = 0; i < n; i++) {
        cout << "\n--- Report " << i + 1 << " ---\n";

        cout << "Enter Location: ";
        getline(cin, reports[i].location);

        cout << "Enter Hazard Type (Tsunami/Flood/High Waves): ";
        getline(cin, reports[i].hazardType);

        cout << "Enter Description: ";
        getline(cin, reports[i].description);
    }

    // Save to file
    ofstream file("reports.txt");
    for (int i = 0; i < n; i++) {
        file << reports[i].location << ", "

```

```

        << reports[i].hazardType << ", "
        << reports[i].description << endl;
    }
    file.close();

    cout << "\nReports saved successfully!\n";

    // Keyword detection
    cout << "\n--- Keyword Analysis ---\n";

    for (int i = 0; i < n; i++) {
        string desc = reports[i].description;

        if (desc.find("tsunami") != string::npos ||
            desc.find("flood") != string::npos ||
            desc.find("wave") != string::npos) {

            cout << "Hazard keyword found in report "
                 << i + 1 << " (" << reports[i].location << ")\n";
        }
    }

    // Hotspot detection
    cout << "\n--- Hotspot Detection ---\n";

    for (int i = 0; i < n; i++) {
        int count = 1;

        for (int j = i + 1; j < n; j++) {
            if (reports[i].location == reports[j].location) {

```

```

        count++;
    }
}

if (count >= 2) {
    cout << "HOTSPOT DETECTED at: "
        << reports[i].location
        << " (Reports: " << count << ")\n";
}
}

cout << "\n--- All Reports ---\n";
for (int i = 0; i < n; i++) {
    cout << "\nLocation: " << reports[i].location;
    cout << "\nHazard: " << reports[i].hazardType;
    cout << "\nDescription: " << reports[i].description << endl;
}

return 0;
}

```

## 9. TEST CASES & OUTPUTS

### Test Case 1: Basic Input (No Hotspot)

#### INPUT

Enter number of reports: 2

Report 1:

Location: Chennai

Hazard: Flood

Description: heavy flood near coastal area

Report 2:

Location: Vizag

Hazard: High Waves

Description: strong wave activity observed

## **OUTPUT**

Reports saved successfully!

--- Keyword Analysis ---

Hazard keyword found in report 1 (Chennai)

Hazard keyword found in report 2 (Vizag)

--- Hotspot Detection ---

(No hotspot detected)

--- All Reports ---

Location: Chennai

Hazard: Flood

Description: heavy flood near coastal area

Location: Vizag

Hazard: High Waves

Description: strong wave activity observed

## **Test Case 2: Hotspot Detection**

### **INPUT**

Enter number of reports: 3

Report 1:



Location: Vizag

Hazard: High Waves

Description: wave levels increasing rapidly

Report 2:

Location: Vizag

Hazard: Flood

Description: flood water entering streets

Report 3:

Location: Chennai

Hazard: Tsunami

Description: possible tsunami warning issued

## **OUTPUT**

Reports saved successfully!

--- Keyword Analysis ---

Hazard keyword found in report 1 (Vizag)

Hazard keyword found in report 2 (Vizag)

Hazard keyword found in report 3 (Chennai)

--- Hotspot Detection ---

HOTSPOT DETECTED at: Vizag (Reports: 2)

--- All Reports ---

...

## **10. TIME COMPLEXITY**

Let  $n$  be the number of reports entered by the user.

### **Input Operation**

The system takes input for each report using a loop that runs  $n$  times.

Time Complexity:  $O(n)$

### **File Writing Operation**

All reports are written to a file using a loop that runs  $n$  times.

Time Complexity:  $O(n)$

### **Keyword Detection**

The system checks each report's description for keywords such as "tsunami", "flood", or "wave". This is done for all  $n$  reports.

Time Complexity:  $O(n)$

### **Hotspot Detection**

The system compares each report with every other report to count occurrences of the same location. This involves nested loops.

Time Complexity:  $O(n^2)$

### **Display Operation**

All reports are displayed using a loop that runs  $n$  times.

Time Complexity:  $O(n)$

### **Overall Time Complexity:**

The most dominant operation is hotspot detection, which takes  $O(n^2)$  time.

Therefore, the overall time complexity of the system is:

$O(n^2)$

## **11. SPACE COMPLEXITY**

### **Data Storage**

The system stores all reports using an array of structures.

Space Complexity:  $O(n)$

### **Auxiliary Variables**

Additional variables such as counters and temporary strings require constant space.

Space Complexity:  $O(1)$

### **Overall Space Complexity:**

The total space required is proportional to the number of reports.

Therefore, the overall space complexity is:

$O(n)$

## **12. ADVANTAGES**

- Demonstrates real-world application of C++ programming
- Provides a simple yet effective approach to disaster data analysis
- Enhances understanding of text processing and data handling
- Can be easily extended into a more advanced system
- Encourages integration of technology with disaster management

### 13. LIMITATIONS

- The system is console-based and lacks a graphical interface
- Social media data is simulated rather than real-time
- Limited scalability for handling very large datasets
- Does not include real-time integration with external systems

### 14. Future Enhancements

The system can be further improved by incorporating advanced features such as:

- Development of a graphical user interface (GUI)
- Integration with real-time social media APIs
- Use of databases like MySQL for efficient data storage
- Implementation of advanced machine learning algorithms
- Real-time map-based visualization of hazard zones

### 15. Conclusion

This project successfully demonstrates how a C++-based system can be used to simulate a disaster reporting and analysis platform. By combining user-generated data and text analysis, the system identifies patterns and potential hazard zones effectively.

Although simplified, the project highlights the importance of integrating crowdsourced data and analytical techniques in disaster management. With further enhancements, this system has the potential to evolve into a powerful tool for improving disaster response and public safety.

### 16. References

1. Havas, C., Resch, B., Francalanci, C., Pernici, B., Scalia, G., Fernandez-Marquez, J. L., ... & Rüping, S. (2017). E2mc: Improving emergency management service practice through social media and crowdsourcing analysis in near real time. *Sensors*, 17(12), 2766.
2. Holderness, T., & Turpin, E. (2016). From social media to geosocial intelligence: crowdsourcing civic co-management for flood response in Jakarta, Indonesia. In *Social media for government services* (pp. 115-133). Cham: Springer International Publishing.
3. Poblet, M., García-Cuesta, E., & Casanovas, P. (2013, December). Crowdsourcing tools for disaster management: A review of platforms and methods. In *International Workshop on AI Approaches to the Complexity of Legal Systems* (pp. 261-274). Berlin, Heidelberg: Springer Berlin Heidelberg.

4. Bono, C., Mülâym, M. O., Cappiello, C., Carman, M. J., Cerquides, J., Fernandez-Marquez, J. L., ... & Pernici, B. (2023). A citizen science approach for analyzing social media with crowdsourcing. *Ieee Access*, 11, 15329-15347.